Manual Técnico de Instalación — API de Asistencia

Plataforma InfoUNSA · Windows 10/11 y Windows Server 2019+ (PowerShell como **Administrador**)

Instala la API de Asistencia (NestJS + Prisma + MySQL) en el puerto **43000** y la deja funcionando como servicio de Windows. Si prefieres Docker, consulta la sección *Instalación con Docker* del `README.md` de la raíz.

Contenido

1. Requerimientos
2. Instalación: a) Clonar · b) Base de datos · c) Archivo `.env` · d) Dependencias · e) Esquema y datos iniciales · f) Compilación y ejecución · g) Regla de Firewall · h) Conversión en un servicio
3. Actualización y respaldos
4. Problemas frecuentes

---

1. Requerimientos

Software	Versión	Descarga
MySQL Installer (MySQL Server)	8.0.46	https://dev.mysql.com/get/Downloads/MySQLInstaller/mysql-installer-community-8.0.46.0.msi
Node.js LTS	24.18.0 (mínimo 22.12)	https://nodejs.org/dist/v24.18.0/node-v24.18.0-x64.msi
pnpm	9 o superior	`npm install -g pnpm` (después de instalar Node.js)
NSSM (*Non-Sucking Service Manager*)	última	https://nssm.cc/download (solo para el método 1 de la sección h)
Git para Windows	2.55	https://github.com/git-for-windows/git/releases/download/v2.55.0.windows.2/Git-2.55.0.2-64-bit.exe

Durante la instalación de Node.js deja marcada la opción **Add to PATH** (necesaria para que el servicio encuentre `node`).

Comprueba las versiones en una terminal nueva (cerrada y abierta después de instalar):

```
node -v     # debe mostrar v22.12 o superior (se recomienda v24.x)
pnpm -v     # 9 o superior
```

> Con una versión antigua de Node (por ejemplo v18) `pnpm install` falla en el paso `prisma generate` con `TypeError: ... styleText is not a function`. Instala Node 24 LTS y repite el paso d.

2. Instalación

> **Antes de empezar:** los pasos a a f funcionan tanto en **CMD** como en **Windows PowerShell**. Los pasos g (firewall) y h (servicio) usan comandos de **PowerShell como administrador** (el indicador empieza con `PS`, por ejemplo `PS C:\>`); si ves `C:\>` sin `PS`, estás en CMD: escribe `powershell` y pulsa Enter. Las rutas de los ejemplos usan `C:\Plataforma\infounsa-asistencia-inventario`; **cámbialas por la carpeta donde clonaste** el repositorio. Si esa carpeta está en otra unidad (por ejemplo `D:`), usa `cd /d D:\ruta` en CMD o `Set-Location D:\ruta` en PowerShell.

a. Clonar el repositorio

En PowerShell o CMD, en una ruta corta y sin tildes (por ejemplo `C:\Plataforma`):

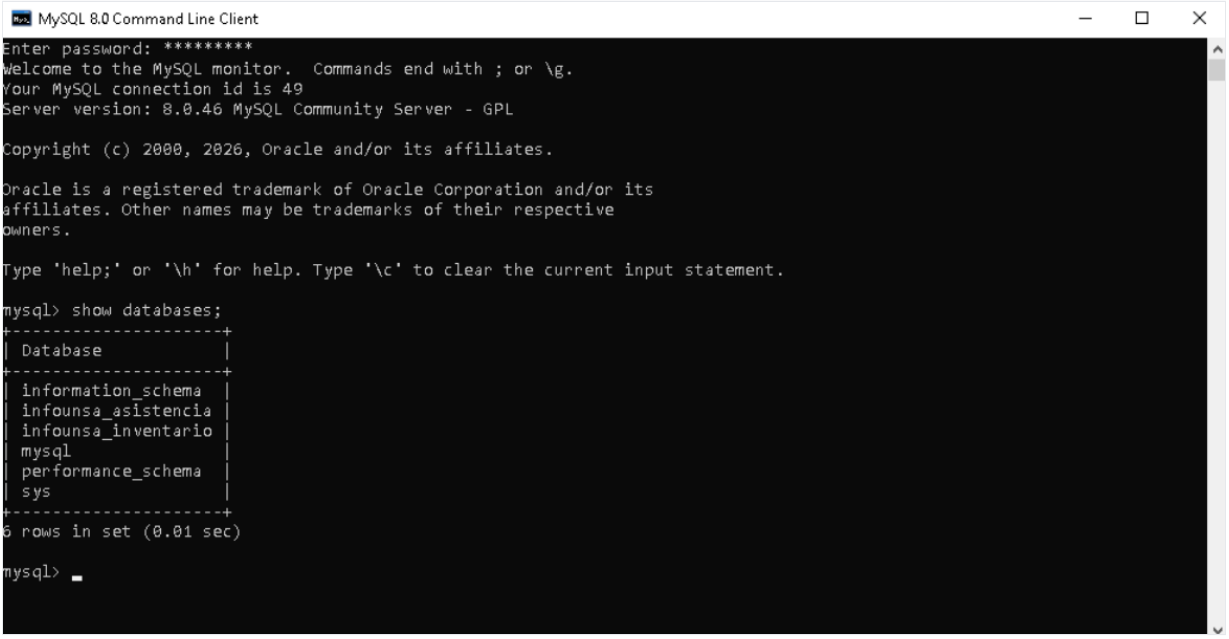
```
cd C:\Plataforma
git clone https://github.com/mtinta/infounsa-asistencia-inventario.git
cd infounsa-asistencia-inventario
```

(Si se te indicó una rama específica: `git fetch origin` y `git switch --track origin/<rama>`.)

b. Configuración de la base de datos

1. Abre **MySQL 8.0 Command Line Client** (o tu gestor de base de datos) e ingresa con la contraseña definida al instalar MySQL.
2. Crea la base de datos vacía y compruébala:

```
CREATE DATABASE infounsa_asistencia CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
SHOW DATABASES;
```



3. Crea un usuario dedicado para las APIs (si ya existe `infounsa` por el otro módulo, omite el `CREATE USER`) y otorga permisos:

```
CREATE USER 'infounsa'@'localhost' IDENTIFIED BY '<contraseña_segura>';
GRANT ALL PRIVILEGES ON infounsa_asistencia.* TO 'infounsa'@'localhost';
FLUSH PRIVILEGES;
```

`infounsa@localhost` es el usuario que recibe los permisos sobre `infounsa_asistencia`. **Anota la contraseña**: se usa en el `.env`.

c. Creación del archivo `.env`

El `.env` guarda la configuración de la API (puerto, conexión a MySQL, secreto de sesión). Debe quedar en la carpeta de la API, con el nombre exacto `.env`. Se crea **antes de instalar las dependencias** (paso d), porque la instalación genera el cliente de Prisma y necesita `DATABASE_URL`.

1. Copia la plantilla que trae el repositorio (en CMD o PowerShell, desde la carpeta donde clonaste):

```
cd <carpeta-del-repositorio>\apps\asistencia\api
copy .env.example .env
```

2. Edítalo con el Bloc de notas:

```
notepad .env
```

Cambia **solo** los valores que dicen `CAMBIAR`:

Variable	Qué poner
`DATABASE_URL`	Sustituye `CAMBIAR_PASSWORD` por la contraseña del usuario `infounsa` que creaste en el paso b. Queda así: `mysql://infounsa:MiClave123@localhost:3306/infounsa_asistencia`
`JWT_SECRET`	Una cadena larga y aleatoria (ver abajo). Sin espacios
`ASISTENCIA_ALLOWED_ORIGINS`	Déjala como está; agrega más orígenes separados por comas solo si una aplicación web consume la API

Guarda con `Ctrl + S` y cierra el Bloc de notas. No agregues comillas, ni espacios alrededor del `=`, y deja cada variable en su propia línea.

3. Genera el secreto aleatorio (necesario: no dejes el valor de ejemplo; copia el resultado y pégalo como valor de `JWT_SECRET`):

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

4. Comprueba que el archivo existe con el nombre correcto:

```
dir .env*
```

Deben aparecer `.env` y `.env.example`. Si ves `.env.txt`, renómbralo: `ren .env.txt .env`.

> **Importante: el Explorador de Windows no puede crear ni renombrar un archivo llamado `.env`** (muestra el error "Debes escribir un nombre de archivo", porque el nombre empieza con punto y no tiene nada antes). Por eso el archivo se crea **desde la consola** con `copy` (y se abre con `notepad .env`), que sí lo permiten. No uses clic derecho > Nuevo > Documento de texto.
>
> Si prefieres usar *Guardar como* en el Bloc de notas: en **Tipo** elige **Todos los archivos (*.*)** y escribe el nombre **entre comillas**: `".env"`. Si no, Windows guarda `.env.txt`. Conviene activar **Vista > Mostrar > Extensiones de nombre de archivo** en el Explorador para ver las extensiones reales.

> **Notas:**
>
> - Si la contraseña de MySQL tiene caracteres especiales (`@`, `:`, `/`, `#`, `%`), escríbelos codificados en la URL (por ejemplo `@` → `%40`, `#` → `%23`) o usa una contraseña sin ellos.
> - No guardes el archivo como "UTF-8 con BOM" (en el Bloc de notas antiguo, no cambies la codificación al guardar): impide leer la primera variable.

Alternativa: crearlo con un solo comando de PowerShell

Solo en **PowerShell** (no en CMD). Reemplaza los valores **sin** los signos `< >`. La línea `@'` debe terminar la línea y `'@` debe ir **al inicio de su línea, sin espacios delante**:

```
cd <carpeta-del-repositorio>\apps\asistencia\api
@'
PORT=43000
NODE_ENV=development
DATABASE_URL=mysql://infounsa:TU_PASSWORD@localhost:3306/infounsa_asistencia
JWT_SECRET=TU_SECRETO_ALEATORIO
ASISTENCIA_ALLOWED_ORIGINS=http://localhost:43000,http://127.0.0.1:43000
'@ | Set-Content .env -Encoding ascii
```

(Con `@'...'@` los caracteres `$` y `` ` `` de la contraseña no se interpretan. `-Encoding ascii` evita el BOM.)

d. Instalación de dependencias

Desde la **raíz** del repositorio (el proyecto es un monorepo con paquetes compartidos, por eso se usa `pnpm` y no `npm install` dentro de la carpeta de la API):

```
cd C:\Plataforma\infounsa-asistencia-inventario
pnpm install
```

> La instalación ejecuta `prisma generate` en cada API y lee `DATABASE_URL` del `.env`; por eso el archivo se crea **antes** (paso c). Si `pnpm install` falla en `prisma generate`, comprueba que el `.env` exista en `apps\<modulo>\api`.
>
> **Resultado esperado:** `pnpm install` debe terminar **sin errores** (puede mostrar avisos `WARN` de paquetes obsoletos, que son normales). Durante la instalación se compilan los módulos nativos (SQLite

cifrado, `bcrypt`, Prisma) y, al final, los paquetes compartidos del repositorio (`packages\*`).

- Si aparece `ERR_PNPM_IGNORED_BUILDS` ("Ignored build scripts"), tu copia del repositorio es anterior a la corrección: actualízala (`git pull`) y repite `pnpm install`. Como alternativa inmediata ejecuta `pnpm approve-builds --all` y luego `pnpm install`.
- Si más adelante la compilación de la API falla con `Cannot find module '@infounsa/shared-config'` o `has no exported member 'Roles'`, compila los paquetes compartidos: `pnpm build:packages` (desde la raíz del repositorio).

El comando instala las dependencias de **todo** el repositorio (ambas APIs y los clientes): si ya lo ejecutaste al instalar el otro módulo, no hace falta repetirlo. Si instalas el segundo módulo después, crea también su `.env` (paso c) y ejecuta `pnpm install` de nuevo solo si la instalación anterior falló.

e. Esquema y datos iniciales

Con las dependencias ya instaladas, entra a la carpeta de la API:

```
cd C:\Plataforma\infounsa-asistencia-inventario\apps\asistencia\api
```

Verifica que el `.env` se lee correctamente (debe mostrar usuario, contraseña, puerto y base):

```
node -e "require('dotenv').config(); console.log(process.env.DATABASE_URL)"
```

Crea las tablas con Prisma:

```
npx prisma db push
```

No ejecutes los scripts de `apps/asistencia/database/init` ni `prisma db pull`: están desactualizados respecto al esquema actual (el seed falla con `The column nombre does not exist`) y `db pull` sobrescribiría `prisma/schema.prisma`. Tampoco uses `prisma migrate deploy`: la migración inicial no es compatible con MySQL.

Crea los datos iniciales (roles, sede, terminal y usuarios). El administrador queda con usuario **`admin`** y contraseña **`admin123`** (también se crea `operador_ast` con la misma contraseña); **cámbiala tras el primer inicio de sesión**. Para usar otra contraseña desde el inicio, agrega `SEED_ADMIN_PASSWORD=<contraseña de 8 o más caracteres>` al `.env` antes de ejecutar el seed:

```
npm run db:seed
```

Datos de demostración: por defecto el seed solo crea roles, sede, terminal y los usuarios `admin` y `operador_ast`. Es seguro volver a ejecutarlo: no cambia contraseñas existentes (para forzarlo, `SEED_RESET_PASSWORDS=true`) ni borra marcaciones. Solo con `SEED_DEMO_DATA=true` (entornos de

> prueba) genera usuarios y marcaciones de ejemplo y **borra todas las asistencias**; no lo uses en producción.

f. Compilación y ejecución del servidor

En `C:\Plataforma\infounsa-asistencia-inventario\apps\asistencia\api`:

1. Elimina compilaciones anteriores (si las hay):

   ```
   npm run clean
   ```

   (Borra la carpeta `dist` y el archivo `tsconfig.tsbuildinfo`; funciona igual en CMD y en PowerShell. Si no existen, no hace nada.)

2. Compila: `npm run build` (equivale a `npx tsc`).

3. Ejecuta el servidor:

   ```
   node .\dist\main.js
   ```

   Debe aparecer `Nest application successfully started` y `Asistencia API escuchando en puerto 43000`.

4. Comprueba en el navegador: `http://localhost:43000/health`. Detén el servidor con `Ctrl + C` antes de continuar con el servicio.

g. Regla de Firewall

Para que otros equipos de la red (los kioscos o aplicaciones de escritorio) lleguen a la API, crea la regla en PowerShell como administrador (cambia el nombre si lo deseas, y el puerto si usas otro):

```
New-NetFirewallRule `
  -DisplayName "Asistencia API 43000" `
  -Direction Inbound `
  -Protocol TCP `
  -LocalPort 43000 `
  -Action Allow
```

Verifica que existe:

```
Get-NetFirewallRule | Where-Object DisplayName -like "*43000*"
```

> No abras el puerto 3306 (MySQL) hacia la red: solo la API debe conectarse a la base.

h. Conversión en un servicio

Un servicio permite que la API arranque sola al encender o reiniciar el servidor y se levante si se cae. Hay **dos métodos**: elige uno.

	Método 1 — NSSM	Método 2 — Script de Windows
Qué crea	Servicio `AsistenciaAPI` (visible en `services.msc`)	Tarea programada `InfoUNSA-Servidores` (corre como `SYSTEM`)
Software extra	NSSM	Ninguno
Si la API se cae	Se reinicia sola a los 5 s	Se reintenta hasta 3 veces
Gestiona	Solo esta API	Asistencia e Inventario juntas

Método 1 — NSSM

1. Abre PowerShell como administrador en la carpeta de NSSM (por ejemplo `C:\nssm\win64`) y ejecuta:

```
.\nssm.exe install AsistenciaAPI
```

2. Se abre una ventana. En la pestaña **Application** configura:

Campo	Valor
Path	`C:\Program Files\nodejs\node.exe`
Startup directory	`C:\Plataforma\infounsa-asistencia-inventario\apps\asistencia\api`
Arguments	`dist\main.js`
Service name	`AsistenciaAPI`

3. Pulsa **Install service**.

Alternativa por línea de comandos (equivalente, sin ventana), desde la carpeta de `nssm.exe`:

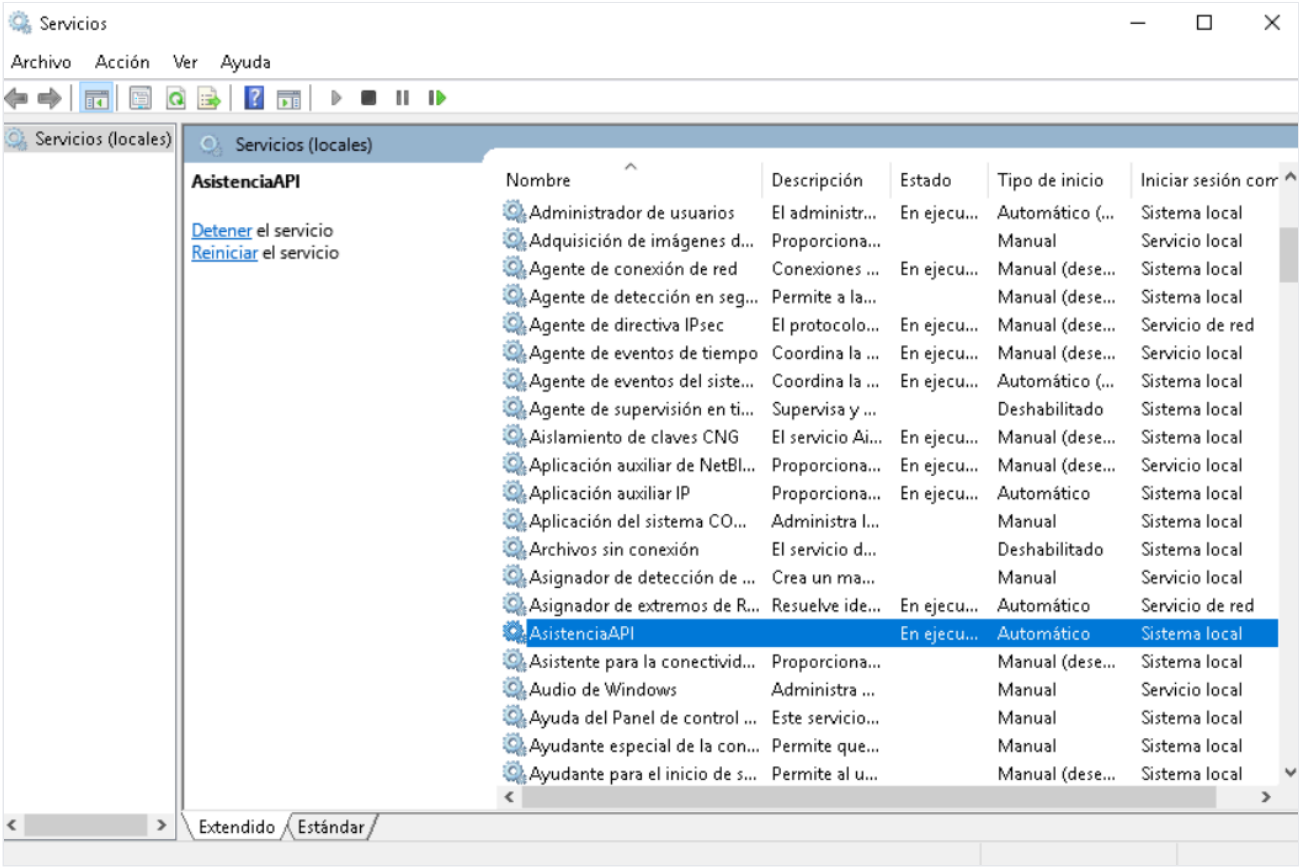
```
$svc = "AsistenciaAPI"
$dir = "C:\Plataforma\infounsa-asistencia-inventario\apps\asistencia\api"
New-Item -ItemType Directory -Force C:\Plataforma\infounsa-asistencia-inventario\.run\logs |
Out-Null
.\nssm.exe install $svc "C:\Program Files\nodejs\node.exe" "dist\main.js"
.\nssm.exe set $svc AppDirectory $dir
.\nssm.exe set $svc Start SERVICE_AUTO_START
.\nssm.exe set $svc AppExit Default Restart
.\nssm.exe set $svc AppRestartDelay 5000
.\nssm.exe set $svc AppStdout "C:\Plataforma\infounsa-asistencia-
inventario\.run\logs\asistencia-api.log"
.\nssm.exe set $svc AppStderr "C:\Plataforma\infounsa-asistencia-
inventario\.run\logs\asistencia-api.err.log"
.\nssm.exe set $svc DependOnService MySQL80
.\nssm.exe start $svc
```

Automático para las dos APIs a la vez, con el script del repositorio:

```
cd C:\Plataforma\infounsa-asistencia-inventario
powershell -ExecutionPolicy Bypass -File scripts\instalar-servicios-nssm.ps1 -NssmPath
C:\nssm\win64\nssm.exe -MySqlService MySQL80
```

Operación del servicio:

```
Start-Service AsistenciaAPI              # o: sc start AsistenciaAPI
Get-Service AsistenciaAPI                # debe indicar Status: Running
Set-Service AsistenciaAPI -StartupType Automatic    # inicio automático (también en
services.msc)
Stop-Service AsistenciaAPI
Restart-Service AsistenciaAPI
```



Con el inicio automático, al reiniciar Windows Server la API arrancará sola.

Quitar el servicio: `.\nssm.exe remove AsistenciaAPI confirm`.

Método 2 — Script de Windows (tarea programada)

No requiere instalar nada. Desde la raíz del repositorio, en PowerShell como administrador:

```
cd C:\Plataforma\infounsa-asistencia-inventario

# Si las bases de datos son un MySQL instalado en Windows (este manual):
powershell -ExecutionPolicy Bypass -File scripts\instalar-autoarranque.ps1 -SinDocker

# Si las bases de datos corren en Docker, omite -SinDocker.
```

El script registra la tarea `InfoUNSA-Servidores`, que al arrancar Windows (con 1 minuto de espera para que MySQL esté listo) inicia **ambas APIs** en segundo plano. Para probarla sin reiniciar:

```
Start-ScheduledTask -TaskName InfoUNSA-Servidores
Get-ScheduledTask   -TaskName InfoUNSA-Servidores
Invoke-WebRequest http://localhost:43000/health
```

Detener: `powershell -File scripts\detener-servidores.ps1`. Quitar la tarea: `powershell -ExecutionPolicy Bypass -File scripts\instalar-autoarranque.ps1 -Quitar`.

> No uses los dos métodos a la vez (los dos intentarían abrir el mismo puerto). Detalles adicionales en el Anexo de servicios de Windows.

Comprobación final (ambos métodos): reinicia el servidor y abre `http://localhost:43000/health` sin iniciar sesión en Windows.

---

3. Actualización y respaldos

Actualizar la API (con el servicio instalado):

```
cd C:\Plataforma\infounsa-asistencia-inventario
git pull
pnpm install
npm run build --prefix apps\asistencia\api
Restart-Service AsistenciaAPI        # Método 2: Stop-ScheduledTask InfoUNSA-Servidores; Start-ScheduledTask InfoUNSA-Servidores
```

Respaldo de la base de datos (programar a diario con el Programador de tareas):

```
mysqldump -u infounsa -p --single-transaction infounsa_asistencia > respaldo-asistencia-AAAAMMDD.sql
```

Restaurar (PowerShell): `Get-Content respaldo-asistencia-AAAAMMDD.sql | mysql -u infounsa -p infounsa_asistencia`.

---

4. Problemas frecuentes

Síntoma	Causa y solución
`P1001: Can't reach database server`	MySQL apagado o `DATABASE_URL` incorrecta (host, puerto, usuario, contraseña).
La API se cierra al arrancar	Revisa `.run\logs\*.err.log`. Suele faltar `JWT_SECRET` o `DATABASE_URL` en el `.env` de `apps\asistencia\api`.
`EADDRINUSE`	El puerto 43000 está ocupado: `Get-NetTCPConnection -LocalPort 43000` y `Stop-Process -Id <PID>`.
El servicio se detiene al instante	Falta el `.env`, no se compiló (`dist\main.js`) o `node` no está en el `PATH` del sistema.
`ERR_PNPM_IGNORED_BUILDS` al ejecutar `pnpm install`	pnpm bloquea los scripts de instalación no aprobados. Actualiza el repositorio (`git pull`) o ejecuta `pnpm approve-builds --all` y repite `pnpm install`
`Cannot find module '@infounsa/shared-config'` / `no exported member 'Roles'` al compilar	Faltan los paquetes compartidos compilados: `pnpm build:packages` en la raíz del repositorio
`styleText is not a function` o `Unsupported engine` al ejecutar `pnpm install`	Node es demasiado antiguo (v18/v20 viejo). Instala Node 24 LTS, abre una terminal nueva, verifica `node -v` y repite `pnpm install`
`Cannot find module ... generated/prisma`	Ejecuta `pnpm install` en la raíz (genera el cliente de Prisma).
Las apps no se conectan desde otros equipos	Falta la regla de firewall (sección g) o el origen no está en `ASISTENCIA_ALLOWED_ORIGINS`.
`The column nombre does not exist`	La base se creó con scripts SQL antiguos. Usa una base vacía y `npx prisma db push`.